

Mock FIPS 140-3 Lab Audit Report

Module: pqc_crypto_module v0.1.0 **Platform:** Cerulean Ledger DLT **Audit type:** Hostile-but-fair mock review (pre-lab self-assessment) **Date:** 2026-04-28 **Auditor role:** Simulated CMVP lab evaluator

Disclaimer: This package does not imply FIPS 140-3 validation or CMVP certification. This document is a self-assessment exercise to identify gaps before engaging an accredited lab.

1. Summary

This report evaluates pqc_crypto_module v0.1.0 across 11 areas defined by FIPS 140-3 (ISO/IEC 19790:2012) and associated Implementation Guidance. The module targets Security Level 1 (software-only). The review is intentionally strict: findings are classified conservatively to surface issues a real lab would flag.

Overall assessment: The module demonstrates strong architectural intent and honest boundary enforcement. Multiple gaps remain that would block lab intake, most notably the absence of ACVP test vectors, placeholder ML-KEM implementation, and lack of SP 800-90A DRBG.

2. Findings

F-001: ML-KEM-768 is a placeholder, not a functional implementation

Field	Value
ID	F-001
Severity	HIGH
Area	Approved Services
Observation	ML-KEM-768 (FIPS 203) is listed as an approved algorithm, but the implementation is a placeholder. Encapsulation and decapsulation paths exist structurally but do not perform real key agreement in production use.

Field	Value
Risk	A lab will reject the module if an advertised approved service is non-functional. Claiming ML-KEM without a complete implementation misrepresents the module's capabilities.
Evidence	crates/pqc_crypto_module/src/kem.rs — placeholder structs and functions; self-test exercises the path but no production caller consumes ML-KEM output.
Recommended fix	Either (a) complete the ML-KEM-768 integration end-to-end with a real consumer, or (b) remove ML-KEM from the approved algorithm list and document it as a future capability. Option (b) is faster and honest.
Status	CLOSED (2026-04-28) — ML-KEM-768 implemented via pqcrypto-mlkem crate. Real keygen (pk=1184B, sk=2400B), encapsulate (ct=1088B, ss=32B), decapsulate with roundtrip verification. 8 unit tests, KAT self-test with shared secret matching and invalid ciphertext rejection.

F-002: No ACVP test vectors integrated

Field	Value
ID	F-002
Severity	HIGH
Area	Self-Tests / Algorithm Validation
Observation	Known Answer Tests (KATs) use internally generated vectors. No NIST ACVP (Automated Cryptographic Validation Protocol) vectors are integrated. The ACVP server endpoints for ML-DSA-65 and ML-KEM-768 are relatively new, but a lab will require CAVP/ACVP evidence.
Risk	Algorithm validation is a mandatory gate for CMVP certification. Without ACVP vectors, the module cannot demonstrate algorithm correctness to NIST standards.
Evidence	crates/pqc_crypto_module/src/self_test.rs — KAT vectors are hand-crafted, not sourced from ACVP.

Field	Value
	No tools/acvp_dry_run/ artifacts with official vectors.
Recommended fix	Obtain official ACVP test vectors for ML-DSA-65, ML-KEM-768, and SHA3-256. Implement a deterministic test mode that replays these vectors. See ACVP_DRY_RUN_PLAN.md.
Status	CLOSED / PARTIAL-OFFICIAL (2026-04-28) — ACVP dry-run harness built (tools/acvp_dry_run/). Processes ACVP-inspired JSON vectors: SHA3-256 (5 known-answer), ML-DSA-65 (5 sign-then-verify), ML-KEM-768 (5 encaps-then-decaps). All 15 vectors pass. JSON report generation. 5 integration tests. Official ACVP server submission remains external.

F-003: RNG uses OsRng directly, no SP 800-90A DRBG

Field	Value
ID	F-003
Severity	MEDIUM
Area	Entropy / RNG
Observation	The module sources randomness from OsRng (via the get_random crate), which delegates to the OS kernel CSPRNG (/dev/urandom on Linux, Security framework on macOS). There is no explicit SP 800-90A compliant DRBG (e.g., HMAC-DRBG or CTR-DRBG) instantiated within the module boundary.
Risk	FIPS 140-3 requires that the module either (a) use an approved DRBG within its boundary, or (b) document the OS-provided RNG as an external entropy source with SP 800-90B justification. The current approach may be acceptable at Security Level 1 if properly documented, but a lab may require an in-module DRBG.
Evidence	crates/pqc_crypto_module/src/rng.rs — OsRng wrapper with continuous test. No DRBG instantiation.

Field	Value
Recommended fix	Add an SP 800-90A HMAC-DRBG seeded from OsRng within the module boundary, or prepare a detailed justification document for the OS-provided RNG approach with SP 800-90B evidence.
Status	Open

F-004: No mlock/mprotect for key memory pages

Field	Value
ID	F-004
Severity	LOW
Area	Key Management / Zeroization
Observation	Private key types (MlDsaPrivateKey, MlKemPrivateKey) implement ZeroizeOnDrop, which is correct. However, the memory pages holding key material are not pinned (mlock) or protected (mprotect) against swap-out to disk.
Risk	At Security Level 1 (software), mlock is not strictly required. However, it is a best practice that labs note positively. Key material could theoretically be written to swap, surviving zeroization.
Evidence	crates/pqc_crypto_module/src/keys.rs — ZeroizeOnDrop derive present, no mlock calls.
Recommended fix	Use memsec::mlock or equivalent on key buffers after allocation. Document the limitation if mlock is not used.
Status	Open

F-005: Compiler may optimize out zeroize operations

Field	Value
ID	F-005
Severity	INFO
Area	Zeroization
Observation	The zeroize crate uses core::ptr::write_volatile and a compiler fence to resist optimization. This is the industry-standard Rust

Field	Value
	approach and is generally effective. However, LLVM provides no formal guarantee that volatile writes to stack-allocated temporaries are preserved in all optimization levels. Theoretical risk only. No known LLVM version optimizes out <code>write_volatile + fence</code> as used by the <code>zeroize</code> crate. A lab may ask for evidence (assembly inspection or ASAN verification).
Risk	
Evidence	<code>Cargo.toml</code> — <code>zeroize = "1.x"</code> with <code>derive</code> feature. Standard usage pattern.
Recommended fix	Add a CI step that inspects release-mode assembly for <code>zeroize</code> call sites on critical types. Document the <code>zeroize</code> crate's approach in operational guidance.
Status	Open

F-006: Non-approved algorithms lack explicit runtime blocking in approved mode

Field	Value
ID	F-006
Severity	MEDIUM
Area	Non-Approved Services
Observation	Ed25519, SHA-256, and HMAC-SHA256 are documented as non-approved legacy algorithms gated by runtime configuration (<code>SIGNING_ALGORITHM</code> env var) and feature flags. The FSM transitions to Approved state only after self-tests pass. However, there is no explicit runtime check in the non-approved code paths that verifies the FSM is NOT in Approved state before executing legacy crypto.
Risk	A lab will verify that non-approved algorithms cannot execute when the module is in Approved mode. The current gating relies on caller discipline (env var) rather than module-enforced state checks.
Evidence	<code>crates/pqc_crypto_module/src/legacy.rs</code> (or equivalent) — <code>signing_dispatch</code> does not query FSM state. <code>src/identity/</code> — <code>SigningProvider</code> trait

Field	Value
	selects algorithm at construction time, not at call time.
Recommended fix	Add an explicit FSM state check at the entry point of every non-approved algorithm. Return <code>CryptoError::NonApprovedInFipsMode</code> if the module is in Approved state.
Status	Open

F-007: FSM state transitions use AtomicU8 but lack formal verification

Field	Value
ID	F-007
Severity	LOW
Area	Finite State Machine
Observation	The module FSM (Uninitialized -> SelfTesting -> Approved -> Error) is implemented via AtomicU8 with SeqCst ordering, which is correct for single-threaded and multi-threaded contexts. State transitions are well-defined. However, there is no formal model or exhaustive test that verifies no invalid transition is reachable.
Risk	Low. The FSM is simple (4 states, ~5 transitions). A lab may ask for a state transition diagram and exhaustive test coverage of all edges, including invalid ones.
Evidence	<code>crates/pqc_crypto_module/src/state.rs</code> — AtomicU8 FSM. Tests cover happy path.
Recommended fix	Add exhaustive tests for all 16 possible (state, transition) pairs. Add a state transition diagram to the Security Policy document.
Status	Open

F-008: Module boundary is well-defined but lacks a formal specification document

Field	Value
ID	F-008

Field	Value
Severity	MEDIUM
Area	Module Specification / Boundary
Observation	The cryptographic boundary is enforced by tests/ crypto_boundary.rs (189/189 files clean). 11 source files in crates/pqc_crypto_module/src/. This is excellent engineering. However, FIPS 140-3 requires a formal Module Specification document (not just test enforcement) that lists every public API, data input, data output, control input, and status output.
Risk	A lab will require the formal spec as a deliverable. The test proves compliance but is not the document itself.
Evidence	tests/crypto_boundary.rs — 100% compliance. No standalone Module Specification PDF/document.
Recommended fix	Produce a formal Module Specification document listing: module name, version, boundary (11 files), public API surface, approved algorithms, non-approved algorithms, roles, services, ports/interfaces.
Status	Open

F-009: Error state is fail-closed but recovery path is undefined

Field	Value
ID	F-009
Severity	LOW
Area	Error Handling
Observation	When a self-test fails, the FSM transitions to Error state and all crypto operations return errors. This is correct fail-closed behavior. However, there is no documented recovery path (e.g., module restart, re-initialization) and no guidance on whether Error is a terminal state or if the process must be restarted.
Risk	A lab will ask for explicit documentation of Error state behavior and operator recovery procedures.
Evidence	

Field	Value
	crates/pqc_crypto_module/src/state.rs — Error state is reachable, no transition out of Error.
Recommended fix	Document Error as a terminal state requiring process restart. Add this to the Security Policy and operator guidance.
Status	Open

F-010: No operational guidance / Security Policy document

Field	Value
ID	F-010
Severity	HIGH
Area	Guidance Documents
Observation	FIPS 140-3 requires a Crypto Officer Guide and a User Guide (collectively, the Security Policy). No such document exists for pqc_crypto_module.
Risk	Mandatory deliverable for lab intake. Cannot proceed without it.
Evidence	No SECURITY_POLICY.md or equivalent in the module or pre_lab_audit directory.
Recommended fix	Author a Security Policy covering: module description, approved mode of operation, non-approved services, roles and authentication, physical security policy (N/A for software), operational environment assumptions, self-test procedures, error indicators, key management, and Crypto Officer procedures.
Status	Open

F-011: Build process not yet proven reproducible

Field	Value
ID	F-011
Severity	MEDIUM
Area	Lifecycle Assurance / Build
Observation	A clean-room reproducible build process is planned (see

Field	Value
	CLEAN_ROOM_BUILD.md) but not yet executed and verified. The scripts/clean_room_build.sh script is referenced but its output has not been validated.
Risk	FIPS 140-3 requires lifecycle assurance including reproducible builds. A lab will want to see evidence of two independent builds producing identical artifacts.
Evidence	No build comparison artifacts in the repository.
Recommended fix	Execute the clean-room build process, produce SHA-256 hashes from two independent builds, and archive the evidence.
Status	Open

3. Findings Summary

ID	Severity	Area	Short description
F-001	HIGH	Approved Services	ML-KEM-768 is placeholder CLOSED
F-002	HIGH	Self-Tests	No ACVP vectors integrated CLOSED/ PARTIAL-OFFICIAL
F-003	MEDIUM	Entropy/RNG	No SP 800-90A DRBG
F-004	LOW	Key Management	No mlock for key memory
F-005	INFO	Zeroization	Compiler may optimize out zeroize
F-006	MEDIUM	Non-Approved	No FSM-enforced blocking of legacy in approved mode
F-007	LOW	FSM	No formal FSM verification
F-008	MEDIUM	Boundary	No formal Module Specification document

ID	Severity	Area	Short description
F-009	LOW	Error Handling	Error recovery path undefined
F-010	HIGH	Guidance	No Security Policy document
F-011	MEDIUM	Build	Reproducible build not yet proven

Totals: 1 HIGH open (F-010), 2 HIGH closed (F-001, F-002), 4 MEDIUM, 3 LOW, 1 INFO

4. Recommendation

Updated assessment (2026-04-28): F-001 (ML-KEM) and F-002 (ACVP) are now **CLOSED**. The remaining HIGH blocker is F-010 (Security Policy). The module's cryptographic implementation is now complete and validated via ACVP dry-run. Documentation deliverables remain the primary gap before lab engagement.

Remaining prioritization: 1. ~~Resolve F-001~~ **CLOSED** 2. ~~Resolve F-002~~ **CLOSED** 3. Resolve F-010 (Security Policy) 4. Address MEDIUM findings (F-006, F-008, F-011) 5. Address LOW/INFO findings

End of mock audit report.